DRAFT — For Review

Tacoma Power Natural Resources

NR Data Delivery Program

Technical Architecture Blueprint

Version 1.0 • 4/20/2026 • Sponsor: Audrey Lamb • Program Lead: Jason Stewart

Prepared for:	NR Data Delivery Team, NR Leadership, UTS Data Engineering
Prepared by:	Jason Stewart, NR Data Delivery Supervisor
Version:	1.0 DRAFT
Date:	April 2026

Acknowledgments

The NR Data Delivery Team extends sincere thanks to Tacoma Water for their generous support and collaboration in building out Natural Resources' technical data blueprint. Their willingness to share lessons learned have been instrumental in shaping this architecture.

1. Purpose

This document defines the technical architecture that will support Tacoma Power Natural Resources (NR) data delivery efforts. It establishes how data moves from the point of collection in the field through to the analysts, science staff, and managers who need it, and it describes the standards the team will follow to keep that pipeline trustworthy, governable, and sustainable.

The architecture is designed for a small, capable delivery team operating within the broader Tacoma Power enterprise environment. It is not a theoretical framework. It reflects the real constraints of the NR program: a greenfield Snowflake environment, 300+ catalogued data nodes across four basins, a mix of paper-to-digital workflows, established field tools like ESRI and PowerApps, and a team mandate to deliver self-service access to authoritative datasets for NR staff.

This blueprint is a working reference, not a requirements document or a procurement justification. It will evolve as the program matures. Sections marked as 'future state' represent capabilities that are planned but not yet in scope for current build phases.

2. Architectural Principles

Four (4) principles guide every design decision in this architecture. They are intended to keep the platform useful, honest, and maintainable for the team that must run it.

2.1 Lineage First

Every dataset must have a traceable origin. NR data begins as a field observation, a sensor reading, or a paper log. The architecture is designed to preserve and surface that lineage from raw file through transformation to mart, so that staff querying Snowflake can trace a number back to the instrument, the staff member, and the date that produced it. Lineage begins the moment a file is dropped into the intake zone, not when it arrives in Snowflake. Lineage is **not** optional; it is the foundation of trust.

2.2 Self-Service by Design

Consumers should be able to find and use data without routing requests through the data team. This means organizing Snowflake marts around NR business domains (not technical layers), publishing clear data dictionaries, and building Tableau and Streamlit applications that expose only what a user needs. The goal is not to expose everything; it is to make the right things easy to find.

2.3 Right-Sized Governance

Governance should be proportional to risk. Authoritative datasets, those feeding FERC articles, WDFW reports, and partnership agreements, require strict ownership, versioning, and access controls. Working files and analytical scratch data require less. The architecture distinguishes between these tiers so that compliance burden does not fall equally on everything.

2.4 Enterprise Alignment

NR does not build parallel infrastructure. All Snowflake schemas, RBAC patterns, GitHub repositories, and ingestion processes operate within enterprise standards governed by UTS Data Services and City of Tacoma ITD. The NR team works within those guardrails and coordinates with UTS before introducing new patterns.

3. Architecture Overview

The NR data architecture consists of seven coordinated layers. Data enters from field collection tools, passes through a structured intake and pipeline process in Snowflake, and is surfaced to consumers through Tableau dashboards, Streamlit applications, and direct SQL access.

The seven layers are:

Layer	Primary Tool(s)	Role in the Pipeline
1. Field Collection	ESRI (Survey123, Field Maps), PowerApps, MS Access (limited)	Captures structured observations, measurements, and events at the point of collection
2. Intake & Drop Zone	SharePoint (_QUEUE folder), Power Automate	Unstructured landing zone where staff deposit files; lineage tracking begins here
3. Detect & Assess	A file watcher, Power Automate	Detects new files, validates structure, assigns canonical lineage ID and governed filename
4. Ingestion & Staging	Python + dbt staging layer + Snowflake	Loads validated files into Snowflake; enforces types, renames columns, flags anomalies
5. Transformation & Modeling	dbt Core (run in Snowflake)	Builds intermediate and mart-layer models; applies business logic and regulatory alignment
6. Self-Service Access	Snowflake (direct SQL)	Exposes curated marts to analysts and power users through governed, discoverable schemas
7. Application & Visualization	Streamlit in Snowflake, Tableau	Internal tools, QA workflows, dashboards, and analytical reports for NR staff

Note: GitHub provides version control across layers 3, 4, 5, and 7. All Python scripts, dbt models, and Streamlit application code live in a single NR repository within the enterprise GitHub organization.

4. Layer 1: Data Collection (ESRI + PowerApps)

NR data begins in the field. The architecture recognizes distinct collection patterns that must feed the pipeline reliably.

4.1 ESRI Field Collection

Survey123 and ArcGIS Field Maps are the preferred tools for structured spatial data collection. These tools are already deployed across several NR groups and produce records that can be pulled directly from ESRI's enterprise portal.

- **Schema alignment**: Field forms must be designed against the target Snowflake staging schema before deployment. A field added to a Survey123 form that has no corresponding column in the staging model will be silently dropped during ingestion. Form design and data model design happen together.

- **Spatial precision**: All ESRI-collected records include geographic coordinates. The pipeline preserves these as a geometry or WKT field in Snowflake so that spatial joins to ESRI layers remain possible downstream.

- **Offline-first**: Field staff frequently operate in areas without connectivity. Survey123's offline mode is supported; records sync on reconnection. The ingestion pipeline must handle duplicate detection for records that sync out of order.

4.2 PowerApps Collection

PowerApps is used for structured data entry workflows that require more complex logic than a simple form, for example multi-step hatchery production records, juvenile separator observations, or field workflows that branch based on prior inputs.

- **Connection method**: Power Automate exports new records to the SharePoint _QUEUE folder on a trigger or schedule; the Python file watcher picks them up identically to a manually deposited file.

  Note: Direct Sharepoint-to-Snowflake connectors are a future-state option.

- **Validation**: PowerApps enforces field-level validation at the point of entry. The intake assess layer applies a second pass (file type, schema conformance, basin/function validation) before a lineage ID is assigned.

4.3 Paper-to-Digital Transition

A meaningful portion of NR data still originates on paper. The architecture does **not** force premature digitization. Instead, it provides two supported pathways for bringing paper records into the pipeline:

a. Staff may reach out to the data team to build a dedicated digitization workflow, such as a structured entry form, that fits the data type and collection cadence.

b. Alternatively, staff may transcribe paper records into a digital format (xlsx or csv) and deposit the resulting file into the _QUEUE folder. The pipeline treats the digital file as the raw source and assigns it a lineage ID exactly as it would any other submission.

Note: The goal is to progressively replace paper workflows with PowerApps or ESRI forms as staff capacity allows. Priority order is driven by regulatory criticality, not convenience. If you are unsure which path to take, contact the data team.

5. Layer 2-3: Data Intake, Drop Zone, and Lineage Assignment

Why We Changed the Intake Paradigm

Previously, the data team worked directly with files wherever staff stored them, navigating shared drives, email attachments, and ad hoc folder structures to locate and retrieve data. While this felt low friction for staff, it created significant problems: the data team had no consistent way to track what had been received, when, or by whom; file versions were ambiguous; and there was no auditable record of what source data underpinned any given number in a report.

The new intake model inverts that relationship. Staff deposit files into a designated intake folder, and the data team's pipeline takes it from there. This shift enables three things that were not possible under the old model:

- Lineage tracking from the moment a file is received, not just after it is processed.
- A complete, auditable log of every file submission, including who submitted it, when, and what happened to it.
- Consistent, automated processing without manual handoff steps that introduce delay and error.

The intake folder is not a filing system. Staff do not need to organize files within it or follow naming conventions. The pipeline handles all of that automatically. Staff responsibility ends at the drop.

5.1 The Drop Zone: _QUEUE

The entry point for all NR data is a single SharePoint library called NR_Data_Intake. Within it, the _QUEUE folder is the only location staff need to know about. It contains one subfolder per basin. Staff drop their file into their basin folder and walk away.

Commented [JS1]: TBD

```
NR_Data_Intake/
  _QUEUE/           ← Staff drop files here. Nothing else required.
    cowlitz/
    cushman/
    nisqually/
    wynoochee/
  _PROCESSING/      ← Managed by the pipeline. Staff do not touch.
  _ARCHIVE/         ← Successfully ingested files, organized by date.
  _ERRORS/          ← Files that failed validation. Alert sent to staff.
```

Staff interact with exactly one folder: their basin's _QUEUE subfolder. All other folders in NR_Data_Intake are managed by the pipeline and require data team permissions to access.

5.2 File Detection and Assessment (Layer 3)

Commented [SJ2]: What about APIs? What will the process be for that.

A (python) file watcher (or Power Automate flow) monitors the _QUEUE subfolders for new files. When a file appears, the assess step runs immediately. This step does not modify the file: it reads it, validates it, and decides whether it can proceed. The assess step checks four things:

Commented [JS3]: Or other file watcher system? Python, Powershell?, FME?

- **File type**: Is the format one the pipeline can process? Accepted formats are xlsx, csv, xml, and json. All other formats are routed to _ERRORS with a clear message to the submitting staff member.
- **Basin match**: Does the subfolder the file was dropped into match a valid NR basin? This is always true if staff used the correct _QUEUE subfolder, but is verified programmatically.
- **Function identification**: The assess step attempts to match the file against the known function-dataset registry (see Section 5.4) based on filename keywords, column headers, or submitter metadata. If a match is found with high confidence, it proceeds automatically. If no match is found, the file is routed to _ERRORS with a request for the staff member to provide function and dataset information.
- **Minimum schema check**: For known dataset types, the assess step checks that required columns are present. Missing required columns route to _ERRORS. Extra columns are accepted and preserved.

5.3 Lineage ID and Canonical Filename Assignment

If a file passes all checks, the pipeline assigns two things before the file moves to _PROCESSING: a lineage ID and a canonical filename. This is the moment lineage begins. The original filename the staff member used is preserved as metadata but is not used in the pipeline.

5.3.1 Lineage ID

Every file entering the pipeline receives a unique lineage ID at the moment it passes assessment. The format is: LIN-{YYYYMMDD}-{NNNNNN}, where NNNNNN is a zero-padded sequential integer. Example: LIN-20251014-000042.

This ID follows the data from the raw file through every Snowflake layer. Every staging record, intermediate record, and mart record carries the lineage ID of the source file that produced it. Staff who question a number in a Tableau dashboard can trace it back to a specific file drop, and from there to the staff member and collection date.

5.3.2 Canonical Filename

Once lineage is assigned, the pipeline renames the file according to a governed naming convention. Staff do not choose this name. The system derives it from the validated basin, function, dataset, and collection date.

Canonical filename pattern: {basin}-{function}-{dataset}-{collection_date}-{lineage_id}.{ext}

Component	Source	Values / Format	Example
{basin}	Drop zone subfolder	cowlitz, cushman, nisqually, wynoochee	cowlitz

{function}	Matched from function registry	HATCH, PASSAGE, ME, SHORE, CR, WATER, LAND	HATCH
{dataset}	Matched from dataset registry (slug)	adult-separator, juvenile-collection, pit-tags, water-temp, etc.	adult-separator
{collection_date}	Extracted from file contents or submitter metadata	YYYY-MM-DD	2025-10-14
{lineage_id}	System-assigned at assess step	LIN-YYYYMMDD-NNNNNN	LIN-20251014-000042
{ext}	Original file extension	xlsx, csv, xml, json	xlsx

Full example: cowlitz-HATCH-adult-separator-2025-10-14-LIN-20251014-000042.xlsx

Note: The original filename submitted by the staff member is preserved in the intake queue log (see Section 5.5) and in the _source_original_filename metadata column in Snowflake. Staff can always look up a lineage ID or canonical filename by searching for the original filename they submitted.

5.4 Function and Dataset Registry

The function-dataset registry is a seed table in dbt (a CSV file maintained in GitHub) that maps known NR dataset types to their canonical basin, function code, and dataset slug. The assess step queries this registry to identify incoming files. New dataset types must be registered before they can be automatically assessed.

Function Name	Function Code	Example Datasets
Hatchery	HATCH	adult-separator, juvenile-collection, broodstock, truck-haul-logs, pit-tags, ozone
Fish Passage	PASSAGE	pit-tag-detections, juvenile-bypass, adult-trap, screw-trap, floating-surface-collector
Monitoring & Evaluation	ME	carcass-sampling, escapement, aerial-survey, fry-stranding, snorkel-survey, creel-survey
Shoreline	SHORE	Property-inventory, asset-survey, debris-monitoring
Cultural Resources	CR	cultural-site-inventory, artifact-survey, tribal-consultation-log
Water	WATER	water-temp, stream-flow, water-quality, lake-levels, pebble-counts, precipitation, sonde-data, wood-placement, gravel-placement, habitat-survey
Land Use	LAND	land-parcel, vegetation-monitoring, , wildlife-survey, forest-inventory, road-management, enforcement

Note: This registry is the authoritative source for function codes and dataset slugs. Any staff member who needs to add a new dataset type contacts the data team, who adds the entry to the registry seed file and deploys the update through the standard GitHub workflow.

Commented [JS4]: Is this Land or Water?

Commented [DL5R4]: Gravel & wood are both placed in water

Commented [DL6R4]: Riverine "habitat survey" may belong in water, w/ another distinction for land based surveys like elk forage, wood ducks etc

5.5 Intake Queue Log

Every file that enters the intake zone, whether it succeeds, fails, or is awaiting assessment, is logged to an intake queue table in Snowflake. This provides the data team with a complete audit trail of all file submissions and their processing outcomes.

Field	Type	Description
lineage_id	VARCHAR	System-assigned lineage ID (null if file failed assessment before ID assignment)
received_at	TIMESTAMP_NTZ	Datetime the file appeared in the _QUEUE folder
basin	VARCHAR	Basin subfolder the file was dropped into
function_code	VARCHAR	Matched function code (HATCH, PASSAGE, ME, SHORE, CR, WATER, LAND)
dataset_slug	VARCHAR	Matched dataset slug from the registry
original_filename	VARCHAR	The filename as submitted by the staff member
canonical_filename	VARCHAR	The system-assigned canonical filename
submitted_by	VARCHAR	SharePoint user identity of the file submitter
file_size_kb	NUMBER	File size in kilobytes at time of receipt
file_hash	VARCHAR	SHA-256 hash of the file for duplicate detection
status	VARCHAR	RECEIVED, VALIDATING, PROCESSING, COMPLETE, ERROR
processed_at	TIMESTAMP_NTZ	Datetime processing completed (null if pending or errored)
snowflake_table	VARCHAR	Target staging table in Snowflake (null if not yet loaded)
record_count	NUMBER	Number of records loaded to Snowflake (null if not yet loaded)
error_message	VARCHAR	Description of validation or ingestion failure (null if successful)

Status lifecycle: RECEIVED → VALIDATING → PROCESSING → COMPLETE / ERROR

5.6 Error Handling and Staff Notification

Files that fail assessment are moved to the _ERRORS folder. A Power Automate notification is sent automatically to the staff member who submitted the file. The notification includes the original filename, the reason for failure, and a plain-language instruction for how to resolve it.

Failure Type	Routing	Notification Message to Staff
Unsupported file format	_ERRORS	"Your file [name] could not be processed because the format is not supported. Please convert to xlsx or csv and resubmit."
Dataset type not recognized	_ERRORS	"Your file [name] could not be matched to a known dataset type. Please contact the data team to register this dataset before resubmitting."

Required columns missing	_ERRORS	"Your file [name] is missing required columns for the [dataset] dataset: [column list]. Please check the data collection template and resubmit."
Duplicate file detected	_ERRORS	"A file with the same content as [name] was already submitted on [date] (Lineage ID: [LIN-...]). If this is a correction, contact the data team."

Important: The data team monitors the _ERRORS folder daily. Files that remain in _ERRORS for more than 5 business days without resolution are escalated to the facility data steward.

Commented [DL7]: Automated?

Commented [SJ8R7]: These are errors, someone needs to review what caused the error and troubleshoot. Or automating a notification that there are errors?

Commented [9R7]: Thinking about automated notice when there is an error instead of daily manual check

6. Layer 4-6: Snowflake Architecture

Snowflake is the central platform for NR data storage, transformation, and self-service access. The NR team operates within enterprise Snowflake governance managed by UTS Data Services. No new Snowflake accounts, enterprise roles, or schema structures are introduced without UTS coordination.

6.1 Logical Schema Structure

NR data is organized into three logical schema tiers within the enterprise Snowflake environment. These map to existing PWR_RAW and PWR schemas and any additional schemas provisioned by UTS.

Schema Tier	Logical Role	Who Accesses It	Example Contents
PWR_RAW.FISH (Staging)	Raw ingested data. Minimally processed. One table per source dataset.	Data team only	stg_cowlitz_hatch_adult_separator, stg_cowlitz_passage_pit_tags
PWR.FISH (Intermediate)	Joined, enriched, business-logic-applied datasets.	Data team, advanced analysts	int_hatch_pit_tag_events, int_hatch_production
PWR.FISH (Marts)	Curated, domain-organized tables for self-service consumption.	Science staff, managers, Tableau, Streamlit	mrt_hatch_releases_annual, mrt_passage_daily

Note: Staging table names follow the canonical filename structure: stg_{basin}_{function}_{dataset}. A file named cowlitz-HATCH-adult-separator-... loads into stg_cowlitz_hatch_adult_separator. The naming convention is consistent end-to-end from drop zone to Snowflake.

6.2 Self-Service Discovery: The Priority Focus

The most common failure mode in data programs is building a well-structured warehouse that no one can navigate. NR has 300 catalogued data nodes across four basins. Without deliberate discoverability design, a staff member logging into Snowflake sees an undifferentiated list of table names and gives up.

6.2.1 Domain-Organized Mart Naming

Mart tables follow the naming convention: mrt_{businessfunction}_{descriptivename}. The business function prefix (hatch, passage, me, water, reg, etc.) groups tables by domain so that users can immediately locate the mart area relevant to their work. Tables within a function are named by biological concept and grain.

Function	Mart Table Name	Description
HATCH	`mrt_hatch_releases_annual`	Annual release numbers by species, facility, and stock
HATCH	`mrt_hatch_broodstock`	Broodstock collection and egg-take records
PASSAGE	`mrt_passage_daily`	Daily fish counts at all monitored passage points
PASSAGE	`mrt_passage_pit_tag_detections`	All PIT tag detection events, joined to release records
ME	`mrt_me_escapement_annual`	Annual escapement estimates by species and reach
ME	`mrt_me_creel_survey_summary`	Creel survey results by basin, species, and survey period
WATER	`mrt_water_temp_daily`	Daily water temperature summaries from all sensors
WATER	`mrt_water_streamflow_daily`	Daily streamflow at all monitored reaches, all basins
REGULATORY	`mrt_reg_ferc_inputs`	Pre-aggregated dataset supporting FERC annual article reporting
REGULATORY	`mrt_reg_wdfw_metrics`	Metrics required for WDFW license reporting

6.2.2 Column-Level Documentation in dbt

Every mart table and every column carries a plain-English description in the dbt schema YAML. This is the data dictionary. It lives in code and is generated automatically into dbt docs. Required fields for every mart column: description, unit (for numeric fields), source system, and data owner.

6.2.3 Streamlit Orientation Dashboards

The data team maintains one Streamlit dashboard per function domain (HATCH, PASSAGE, ME, WATER, etc.). These are orientation tools, not analytical dashboards. Each dashboard shows what mart tables are available in that domain, when each was last refreshed, known data gaps or quality flags, and the lineage IDs of the most recently ingested source files. Streamlit dashboards are hosted in Snowflake (Streamlit in Snowflake) and are accessible to all NR Viewer-tier users without additional tooling.

6.2.4 Standardized Metadata Columns

Every mart table includes a consistent set of metadata columns so users can assess data currency and provenance without reading external documentation.

Column Name	Type	Description
`_lineage_id`	VARCHAR	The LIN-YYYYMMDD-NNNNNN ID of the source file that produced this record. Links back to the intake queue log.
`_source_basin`	VARCHAR	Originating basin (cowlitz, cushman, nisqually, wynoochee)
`_source_function`	VARCHAR	Function code of the source dataset (HATCH, PASSAGE, ME, SHORE, CR, WATER, LAND)
`_source_facility`	VARCHAR	Originating facility group (e.g., CSH, CFFF, Cushman)
`_canonical_filename`	VARCHAR	The system-assigned canonical filename of the source file
`_ingested_at`	TIMESTAMP_NTZ	When the source file was loaded into Snowflake staging
`_transformed_at`	TIMESTAMP_NTZ	When the mart model last ran and processed this record
`_data_owner`	VARCHAR	NR staff member or role accountable for the accuracy and currency of this dataset. Distinct from the data steward, who may be responsible for day-to-day collection.
`_regulatory_flag`	BOOLEAN	TRUE if this record feeds a regulatory deliverable
`_quality_status`	VARCHAR	PASSED, FLAGGED, or REVIEW: set by dbt data tests

6.2.5 Consumer Roles and Tiered Access

Role Tier	Who It Covers	Access Level	Use Case
Analyst	All NR staff with a Snowflake login	SELECT on all PWR.FISH mart tables	Query pre-built marts, connect Tableau, run exploratory SQL, Build derived queries, cross-domain analysis, validate dbt outputs
Creator	Data delivery team members	Full access to staging, intermediate, marts + write access for ingestion service accounts	Pipeline development, model deployment, QA

6.3 Materialization Strategy

Materialization refers to how dbt persists query results in Snowflake. Choosing the right materialization for each model is one of the most impactful decisions in the pipeline, directly affecting query performance, compute cost, and data freshness. The NR pipeline uses four materialization types, each applied to the model layer where it is most appropriate.

Views

Staging models are materialized as views by default. A view does not store data; it executes the underlying SQL at query time. This is appropriate for staging because: (1) staging models are rarely queried directly by end users, (2) they are lightweight transformations that add little overhead, and (3) keeping them as views avoids duplicating raw data in Snowflake storage. The tradeoff is that each query re-executes the transformation logic, so views are not appropriate for models that are queried frequently or that involve heavy computation.

Tables

Mart models (mrt_) and regulatory models (reg_) are always materialized as tables. Tables store the result of the model query as a physical object in Snowflake, which means queries against them are fast regardless of the complexity of the underlying logic. Full-refresh table models drop and recreate the entire table on each dbt run. This is appropriate for mart tables that represent slowly-changing annual summaries or reference-aligned aggregations where completeness matters more than incremental efficiency. Regulatory tables always use full refresh to ensure that any correction to upstream data is reflected in the authoritative output.

Incremental Models

High-frequency datasets use incremental materialization. Incremental models insert or update only new or changed records on each run, rather than rebuilding the entire table. This is critical for datasets like daily sensor readings, daily separator counts, or PIT tag detection events where the table grows continuously and a full rebuild on every run would be computationally expensive.

The incremental strategy for NR models uses a unique key (typically a composite of lineage_id + record-level identifier) combined with an updated_at timestamp filter. On each run, dbt identifies records with a lineage_id newer than the most recent run and merges them into the existing table. This approach handles late-arriving files (a file submitted today for data collected last week) correctly because the filter is on lineage timestamp, not collection date.

> *Note: Incremental models must be rebuilt from scratch (using --full-refresh) whenever the model's schema changes, such as adding a new column. Scheduled full-refresh runs for incremental models should be planned for low-traffic windows.*

Snapshots

For slowly-changing dimensions such as the facility registry, species code table, and stewardship assignments, dbt snapshots are used to capture historical state. Snapshots record changes over time by adding dbt_valid_from and dbt_valid_to timestamps to each row. This allows the pipeline to reconstruct what a reference table looked like at any point in the past, which is important for regulatory audits where the configuration in effect at the time of data collection may differ from the current configuration.

Clustering

Large fact tables, specifically PIT tag event tables, daily water temperature tables, and daily streamflow tables, are clustered on (date, basin). Clustering physically organizes micro-partitions in Snowflake so that time-range and basin-scoped queries skip large portions of the table entirely. For tables with

millions of rows spanning multiple years and all four basins, clustering can reduce query times by an order of magnitude for the most common access patterns.

6.4 Data Quality and Testing

Test Category	Examples	Applied At
Schema tests	not_null, unique, relationships, accepted_values	Staging and mart layers
Domain tests	Species code in valid list, count >= 0, temperature within sensor range	Intermediate and mart layers
Regulatory tests	FERC article dataset completeness, reporting period coverage	Regulatory mart tables only
Freshness tests	Source file arrived within expected window, no stale staging records	Staging layer

Note: Failed tests do not halt pipeline runs for operational data. Records that fail tests are flagged with _quality_status = FLAGGED and an alert is sent to the data owner. For regulatory data, test failures trigger a hold and require data team review before the record advances to the mart layer.

7. Layer 5: dbt Transformation Architecture

dbt Core is the transformation engine for the NR data pipeline. It runs inside Snowflake and all models are stored in the NR GitHub repository and executed via Snowflake tasks on a scheduled basis.

7.1 Layered Model Structure

Layer	Prefix	Description	Key Rules
Staging	`stg_`	One model per source file type. Standardizes columns, enforces types, preserves lineage ID. No joins.	Views. Column names snake_case. Naming: stg_{basin}_{function}_{dataset}.
Intermediate	`int_`	Cross-source joins, enrichment, species/facility lookups.	No heavy aggregation. No scenario logic. Feeds multiple marts.
Marts	`mrt_`	Domain-organized, consumer-ready, documented tables.	Always materialized as tables. All _metadata columns present. Named mrt_{businessfunction}_{name}.

Regulatory	`reg_`	Pre-aggregated inputs for FERC articles and WDFW reports.	Separate prefix. Stricter test requirements. Change review required.
Write-back	`wrb_`	SME override inputs from Streamlit. Read by dbt, never written by dbt.	Audit fields required. Merged into mart and regulatory models.

7.2 Naming Conventions

- Staging model names: stg_{basin}_{function_lower}_{dataset_slug}. Example: stg_cowlitz_hatch_adult_separator. This mirrors the canonical filename exactly, making it trivial to trace a staging table back to its source files.

- Mart model names: mrt_{businessfunction}_{descriptivename}. The business function prefix groups mart tables by domain. Example: mrt_hatch_releases_annual, mrt_passage_daily, mrt_me_creel_survey_summary, mrt_water_temp_daily, mrt_reg_ferc_inputs.

- Column names: snake_case throughout. Dates as {concept}_date. Counts as {concept}_count. Temperatures as {concept}_temp_c.

7.3 Reference and Lookup Tables (Seeds)

The NR domain depends on a rich set of shared reference data. These are maintained as seed files in dbt: CSV files committed to GitHub that are loaded into Snowflake as reference tables on each dbt run. Seeds are the authoritative source of truth for NR classifications and configurations. They must be reviewed and updated through the standard GitHub pull request workflow before any change takes effect in the pipeline.

The following seed files are required for the NR pipeline to operate correctly:

Seed File	Key Fields	Purpose
`seed_function_dataset_registry.csv`	function_code, dataset_slug, basin, required_columns	Drives the assess step. Maps incoming files to their canonical function and dataset type. Every dataset type the pipeline can ingest must have a row here.
`seed_species_codes.csv`	species_code, common_name, scientific_name, wdfw_code, tribal_code	Canonical species reference. Used by intermediate models to validate and standardize species identifiers across all basins and data collection systems.
`seed_facility_registry.csv`	facility_id, facility_name, basin, function_code, latitude, longitude, active_flag	All NR facility locations across four basins. Used for spatial joins, basin validation, and facility-level aggregations in mart models.

`seed_basin_registry.csv`	basin_code, basin_name, hydrologic_unit, license_number	Basin metadata including hydroelectric license identifiers. Used in regulatory models to link basin-level data to FERC article obligations.
`seed_reg_mapping.csv`	article_number, article_title, basin, reporting_cadence, due_month, required_datasets	Maps each FERC license article to its reporting requirements, applicable basin, and the NR datasets that feed its deliverables. Drives reg_ model test coverage.
`seed_data_owners.csv`	dataset_slug, basin, data_owner, data_steward, steward_email, last_reviewed	Maps each dataset to its data owner and data steward. Powers the _data_owner metadata column on mart tables and drives error notification routing.
`seed_intake_accepted_formats.csv`	file_extension, mime_type, notes	Authoritative list of file formats accepted by the intake pipeline. Used by the assess step to route unsupported formats to _ERRORS without hardcoding format lists in application code.
`seed_ref_reporting_calendar.csv`		Supports seasonal grouping, reporting windows, and period-based aggregation logic
`seed_ref_units_values.csv`		Standardized units, conversions, and plain English value labels for user-facing delivery

Note: Adding a new dataset type to the pipeline requires updating seed_function_dataset_registry.csv and seed_data_owners.csv at minimum. Changes to seed_ferc_articles.csv or seed_wdfw_reporting_thresholds.csv require review by the data team and the relevant license coordinator before merging.

8. Layer 6: Streamlit Application Layer

Streamlit in Snowflake (SiS) is used for lightweight internal tools that support NR staff workflows beyond what a Tableau dashboard provides. Key planned applications:

Application	Primary User	Purpose

Intake Status Monitor	Data team, facility contacts	Show per-basin ingestion status: last file received, lineage ID, processing outcome, any _ERRORS requiring attention
Data QA Review Tool	Data owners, science staff	Surface flagged records by domain and allow owners to approve, reject, or annotate quality flags
Regulatory Dataset Viewer	License coordinators, science staff	Browse and export pre-aggregated FERC article inputs with clear lineage chain displayed
Hatchery Production Override	Hatchery managers	Submit corrections to hatchery production records with mandatory justification fields (write-back to wrb_ tables)
Domain Orientation Dashboards	All NR staff	One dashboard per function domain. Shows available mart tables, last refresh timestamps, known data gaps, and quality flags. Replaces Snowsight dashboards.

Any Streamlit application that allows a user to modify data writes to a designated wrb_ table in Snowflake, never directly to a staging or mart table. Write-back tables include mandatory audit fields: user_id, submitted_at, override_reason, and the lineage_id of the source record being corrected.

9. Version Control (GitHub)

A single GitHub repository within the enterprise organization contains all NR data delivery code.

```
nr-data-delivery/
  dbt/
    models/
      staging/          # stg_{basin}_{function}_{dataset} models
      intermediate/     # int_ models
      marts/            # mrt_ models by domain
      regulatory/       # reg_ models
      write_back/       # wrb_ read models
    seeds/              # All reference seed CSVs
    tests/              # Custom dbt tests
  ingestion/            # Python file watcher, assess/assign, SharePoint ->
Snowflake
  streamlit/            # Streamlit application code
  docs/                 # Architecture docs, data dictionaries
```

The team uses a two-branch model: main (production) and develop (active work). Engineers use feature branches named {initials}/{feature-slug} and open pull requests to develop before merging. dbt model runs and ingestion scripts are triggered via Snowflake tasks on a daily schedule.

10. Visualization (Tableau)

Tableau is the primary visualization layer for NR data consumers. All Tableau workbooks connect to Snowflake mart tables only, never to staging or intermediate layers, and never directly to SharePoint files.

- **Published data sources**: Each mart domain has a corresponding published Tableau data source. Workbooks connect to published sources so that when the underlying mart table changes, all dependent workbooks refresh automatically.

Commented [DL10]: published "tableau data source" or published to snowflake prod?

- **Domain workspaces**: Tableau Server is organized into NR function domain projects: HATCH, PASSAGE, ME, WATER, and Regulatory. Users navigate to their domain project, not a flat list of workbooks.
- **Self-service development**: Analyst-tier users can connect Tableau Desktop to published data sources and publish to a designated NR Self-Service Sandbox project rather than to the governed domain projects.

10.1 Phase 1 Workbook Set

Workbook	Domain	Key Metrics
Fish Passage Overview	PASSAGE	Daily passage counts by species and facility, PIT tag efficiency, bypass flow compliance
Hatchery Production Summary	HATCH	Egg takes, rearing survival, release numbers by stock and year
Water Temperature Monitor	WATER	Daily temperature by sensor location, exceedance alerts, seasonal trends
Streamflow Compliance	WATER	Minimum instream flow compliance by reach, FERC threshold tracking

11. Data Governance Foundations

11.1 Data Stewardship

Every mart table has a named data steward: an NR staff member who is the first point of contact for questions about accuracy, collection methodology, and known gaps. Data stewards are also the recipient of _ERRORS notifications for their facility's datasets. Steward assignments are tracked in the dbt seed_data_owners seed file and surfaced through the _data_owner metadata column on each mart table.

11.2 Authoritative Dataset Certification

Datasets that feed regulatory deliverables or are operationally critical are designated as Authoritative Datasets. Authoritative certification requires: a named data owner, a documented collection methodology, at least 30 days of successful intake queue history, a defined refresh cadence, and

passing dbt tests with no open FLAGGED records. The Authoritative Dataset Registry drives prioritization order for pipeline build.

11.3 Change Management

Changes to mart table schemas or mart-level business logic require a documented pull request reviewed by at least one other data team member. Breaking changes to tables consumed by Tableau published data sources require 5 business days notice to affected workbook owners. The function-dataset registry (Section 5.4) follows the same review process: new entries require a pull request, not an ad-hoc edit.

12. Open Questions and Decisions Required

#	Question	Owner	Impact If Unresolved
1	What are the naming conventions we must follow?	UTS Data Services	Staging and mart models cannot be deployed until schema names are confirmed
2	Can the Python file watcher run as a scheduled task on a UTS-managed server, or must it be implemented entirely via Power Automate?	UTS Data Services	Affects the assess/assign architecture for the intake layer
3	Will PowerApps Sharepoint-to-Snowflake connectors be available, or must PowerApps output route through SharePoint _QUEUE?	UTS / ITD	Affects PowerApps ingestion path

Appendix A: Glossary

This glossary defines terms used in this blueprint and in the broader NR data delivery program. It is written for NR staff who may be encountering these concepts for the first time as the data platform is rolled out.

Term	Definition
_QUEUE	The SharePoint drop zone subfolder where staff deposit raw data files. The only SharePoint location staff need to interact with.
Assess Step	The automated process that runs when a file lands in _QUEUE. It checks file type, matches the file to a known dataset type, validates that required columns are present, and either assigns a lineage ID or routes the file to _ERRORS.
Canonical Filename	The system-assigned filename given to a file after it passes assessment. Format: {basin}-{function}-{dataset}-{collection_date}-{lineage_id}.{ext}. Replaces the original filename inside the pipeline. Staff's original filename is preserved in metadata.
Core Dataset	Any dataset critical for regulatory compliance, operations, or long-term science. Core datasets require a named data owner, documented methodology, and passing dbt tests.
dbt (data build tool)	The transformation framework used by the NR data team to build and test Snowflake models. dbt lets the team write SQL models as code, test them automatically, and document them in a shared catalog. Staff do not interact with dbt directly, but its outputs are the mart tables they query.
Data Owner	The NR staff member or role accountable for the overall accuracy, completeness, and appropriate use of a specific dataset. The data owner defines what 'correct' looks like and is the escalation point for unresolved quality issues. Different from the data steward, who handles day-to-day collection.
Data Steward	The NR staff member responsible for the day-to-day accuracy of a specific dataset at the point of collection. The data steward ensures files are submitted correctly and responds to error notifications. The data steward is often a field technician or facility lead.
Data Slug	A short, lowercase, hyphenated identifier for a specific dataset type within a business function. Slugs are defined in the function-dataset registry and become part of the canonical filename and staging table name. Examples: adult-separator, water-temp, creel-survey. Slugs are machine-friendly identifiers, not display names.
Dataset	A specific type of data collected by NR staff for a specific purpose. For example, 'adult separator observations at the Cowlitz hatchery' is a dataset. Each dataset has a slug, a function code, and a staging table in Snowflake.
Function Code	A short code identifying the NR business function that produced a dataset: HATCH, PASSAGE, ME, SHORE, CR, WATER, or LAND. Function codes are used in canonical filenames, staging table names, and mart table prefixes.
Function-Dataset Registry	A dbt seed file (maintained in GitHub) mapping known NR dataset types to their canonical basin, function code, and dataset slug. The assess step queries this registry to identify incoming files automatically.

Incremental Model	A dbt model that only processes new or changed records on each run, rather than rebuilding the entire table. Used for high-frequency datasets like daily sensor readings. Incremental models are faster and cheaper to run than full-refresh models for large, continuously growing tables.
Intake	The process of receiving a data file from staff, validating it, assigning a lineage ID, and loading it into Snowflake staging. The intake pipeline is the entry point to the NR data platform.
Lineage ID	A unique system-assigned identifier (format: LIN-YYYYMMDD-NNNNNN) assigned to every file when it passes intake assessment. Follows the data through every layer of the pipeline, from the raw file to the mart table. Allows any number in any report to be traced back to a specific file submission.
Mart Table	A curated, consumer-ready Snowflake table built by dbt. Named by business function and biological concept (mrt_{businessfunction}_{name}). The primary target for Tableau connections and self-service SQL queries. Mart tables are always documented, always tested, and always include standard metadata columns.
Materialization	How a dbt model is stored in Snowflake. Options include view (executes at query time, no storage), table (stored result, fast to query), incremental (appends new records), and snapshot (tracks historical state over time). The NR pipeline uses all four depending on the model's role and query frequency.
Model (dbt)	A SQL file in the dbt project that defines a transformation. Each model produces one table or view in Snowflake. Models are the building blocks of the NR pipeline: staging models clean raw data, intermediate models join and enrich it, and mart models produce the consumer-ready output.
Partnership Agreements	Binding regulatory and operational commitments made by Tacoma Power Natural Resources to agencies such as WDFW and tribal co-managers as part of hydroelectric license conditions and negotiated settlements. Partnership agreement compliance metrics are a primary driver of data quality and reporting requirements in the NR data platform.
Pipeline	The end-to-end automated process that moves data from a file dropped in _QUEUE through assessment, ingestion, transformation, and into mart tables available for reporting. Staff interact with the beginning of the pipeline (dropping files); the data team builds and maintains the rest.
RBAC (Role-Based Access Control)	The system that determines who can see and do what in Snowflake. NR uses three tiers: NR Viewer (read mart tables), NR Analyst (read intermediate + mart), and NR Data Team (full access). RBAC is managed by UTS in coordination with the NR data team.
Seed (dbt)	A CSV file committed to the GitHub repository and loaded into Snowflake as a reference table on each dbt run. Seeds are the authoritative source for NR reference data: species codes, facility identifiers, function codes, and partnership agreement reporting thresholds. Changes to seeds go through the standard pull request review process.
Schema	A logical container in Snowflake that groups related tables together. NR uses PWR_RAW.FISH for raw staging data and PWR.FISH for intermediate and mart tables. Think of a schema like a folder inside a database.
Self-Service	The ability for NR science staff and managers to query mart tables, connect Tableau, and explore data without routing requests through the data team. Self-service is a core design goal of the NR data platform.

Snowflake	The cloud data warehouse used by NR to store, transform, and serve data. Snowflake is the system of record for all processed NR data. Staff with NR Viewer access can query mart tables directly using SQL or through Tableau and Streamlit.
Staging (Layer / Table)	The first layer of processed data in Snowflake. Staging tables (prefix: stg_) hold raw data as received from staff files, with minimal transformation: standardized column names, enforced data types, and an attached lineage ID. Staging tables are not intended for direct staff use.
Streamlit in Snowflake (SiS)	A Snowflake-hosted application platform used by the NR data team to build lightweight internal tools. Streamlit dashboards and tools are accessible to NR staff through a browser without needing to install software. NR uses Streamlit for orientation dashboards, QA tools, regulatory viewers, and data correction workflows.
Tableau	The primary data visualization tool used by NR staff. Tableau connects to Snowflake mart tables through published data sources and displays pre-built reports and dashboards. NR Analyst-tier users can also build their own workbooks in Tableau Desktop.
Write-Back	A record submitted by NR staff through a Streamlit application to correct or annotate a data record. Write-backs go to designated wrb_ tables in Snowflake and are merged into mart models by dbt. They are never written directly to staging or mart tables, preserving a clean audit trail.

Appendix B: Staff Data Intake Guide

This guide is written for NR staff submitting data files to the data platform. You do not need to understand the technical pipeline to use it. You just need to know where to drop files and what to include when you notify the data team.

What Is the Intake Folder?

The Intake folder is your delivery point for data going to the Natural Resources Data Platform. Think of it as a loading dock: you do your day-to-day work wherever you normally do it, and when you have data for the data team you drop it here.

Once a file is in Intake, the data team takes ownership. You do not need to rename it, reformat it, or organize it. Drop it exactly as it is.

> The One Rule
>
> Do not rename, edit, or reorganize files before dropping them. The original filename is part of the data record, logged exactly as-is so we can trace the file back to its source. A file called "Temp Oct 15 FINAL johns copy.csv" is fine. Drop it.

How To Submit a File

1. Find your basin folder. Open the folder for your basin: COWLITZ, CUSHMAN, NISQUALLY, or WYNOOCHEE.
2. Find your business function folder. Inside your basin are folders, one for each business function. Open the one that matches your work area. See the reference table below if you are unsure.
3. Drop the file. Copy or move your file into the folder. Do not rename it. Do not create new sub-folders.
4. Notify the data team. Send a quick Teams message or email so we know a file has been dropped. See 'What To Include' below.

Your Folder Structure

Basin	Folder	Business Function	Includes
COWLITZ	HATCH	Hatcheries	All hatchery facilities in basin
	PASSAGE	Fish Passage	Collector, ladder counts
	ME	Monitoring & Evaluation	Creel, SGS, fry stranding
	LAND	Wildlife and Lands	Forestry, enforcement, roads, wildlife
	SHORE	Shoreline	Condition, vegetation, encroachment

	CR	Cultural Resources	Site inventory, monitoring, compliance
	HABITAT	Habitat	Channel surveys, large woody debris, restoration
	WATER	Water	Flow, productivity, WQ, temperature
	REG	Regulatory	Env. permitting, FHC, FTC, FERC
CUSHMAN	same folders	All Cushman basin programs	
NISQUALLY	same folders	All Nisqually basin programs	
WYNOOCHEE	same folders	All Wynoochee basin programs	
SHARED	PTAGIS, LAKE_LEVELS, WEATHER	Cross-basin data only	PTAGIS, USGS gauges, regional weather. If your data is basin-specific, use your basin folder.

What To Include in Your Notification

When you notify the data team, please include:

- What the file contains (e.g., "Creel survey data from Riffe Lake, summer 2024")
- Date range (e.g., "June through September 2024")
- Source: instrument export, manual entry, lab system, legacy archive, etc.
- Any known quirks (e.g., "First 10 rows are header junk" or "August data is missing")
- One-time or ongoing? Historical file or regular delivery?

You do not need a formal write-up. A quick Teams message is fine.

Accepted File Formats

Format	Examples	Notes
Excel	.xlsx, .xls	Most common. Drop as-is, all sheets included.
CSV	.csv, .txt (delimited)	Preferred for sensor and logger exports.
JSON / XML	.json, .xml	Accepted for structured exports from field tools.
Other	Access, PDF, proprietary exports	Check with the data team first.

What Happens After You Drop a File

Once a file lands in Intake, the data team pipeline will:

5. Log it in the data registry with a unique lineage ID tied to your original filename.
6. Validate its format and match it to the correct dataset type.

7. Move it to the permanent data repository under a standardized canonical name. Your original file is preserved exactly as dropped.
8. Clean and standardize it through the processing pipeline.
9. Load it into Snowflake where it becomes available for reporting and analysis.
10. Follow up if there are any questions or issues.

You can ask for a status update at any time by referencing the filename or the date you dropped it. If your file ends up in _ERRORS, you will receive an automatic notification explaining what needs attention.

Questions?

Contact the data team. We would rather answer a quick question than have data sitting in the wrong folder.